AI STATION

EXTREME IMPLEMENTATION SPECIFICATION

Complete simulation, machinery, submarine, UI, progression, architecture, formulas, failure chains, save/load, and test requirements for an AI game creator.

Core identity: A fictional AI-driven survival-management simulation. The AI is the station itself. The player operates, repairs, upgrades, and expands the station while an autonomous submarine explores open water, gathers resources, and returns them for processing.

Visual north star: a believable industrial facility shown as a continuous side-on cutaway, like a compact Fallout shelter cross-section. Thick outer hull, rooms, floors, bulkheads, access paths, pipes, cables, platforms, tanks, generators, processing equipment. Not a row of clickable buildings.

Implementation rule: UI reflects authoritative simulation state. No fake progress bars, no free resources, no buttons that only change labels, and no untested claims of completion.

Version 1.0 • Mobile-friendly 2D • Side view • Portrait-friendly • Single-player/offline

DOCUMENT MAP

SECTION	CONTENT
01	Executive summary and hard constraints
02	Vision, player role, objectives, gameplay loop
03	Visual composition and station cutaway architecture
04	Coordinates, simulation clock, update order
05	Common module framework and machine state machine
06	Station modules and machine behavior
07	Resources, inventory, units, and recipes
08	Autonomous submarine systems and AI
09	Underwater world, depth, pressure, hazards, deposits
10	Economy, progression, upgrades, balancing
11	Damage, wear, faults, maintenance, cascading failures
12	UI, interactions, alerts, accessibility
13	Data model, pseudocode, validation
14	Save/load, determinism, performance
15	Starting values and balancing guardrails
16	Testing and acceptance criteria
17	Milestones and scope control
18	Exact instructions for the AI creator

Data-driven content is mandatory: recipes, resources, modules, upgrades, depth bands, and fault rules belong in editable definitions, not hard-coded across unrelated UI handlers.

01. Executive Summary and Hard Constraints

AI Station is a fictional survival-management game about operating a remote autonomous ocean station. The station itself is the AI. There is no human crew-management loop and no need for living quarters. The player sets priorities, maintains systems, develops resource chains, researches improvements, and sends an autonomous submarine deeper into the ocean.

Must preserve

• Portrait-friendly 2D side view: station at top, clear ocean surface, open underwater descent below.

• Station is one coherent cutaway facility with continuous hull, floors, rooms, bulkheads, corridors, machinery footprints, pipes, cables, platforms, ladders, hatches, and plausible access.

• Underwater center stays mostly open for the sub. Fixed cliff/reef edges, occasional ledges, fish, wreck silhouettes, coral, and deposits provide context without cluttering the travel lane.

• Looping vertical background uses reusable tiles and parallax: distant silhouettes, midground ruins, foreground cliffs.

• One small autonomous submarine with tiny articulated arms scans, grabs, and mines resource nodes on cliff faces and seabed.

• Depth has real consequences: pressure rises; hull rating, battery, tools, cargo, power, and upgrades affect mission reach and safety.

• Organic material can be captured and processed into bio-nutrients or industrial inputs. The sub collects minerals, salvage, and unusual samples. Refining converts inputs into fuel/oil, components, and upgrades.

• Selecting a module/resource opens an inspection panel with status, condition, inputs/outputs, power, work, faults, and actions.

• Do not add human crew, living quarters, combat, online multiplayer, top-down ocean map, photorealism, or a 3D redesign.

Forbidden shortcuts

• No decorative progress that does not correspond to simulation progress.

• No resources from nowhere except starter stock, explicit rewards, or defined renewable sources.

• No production without valid inputs, power, and output capacity.

• No silent resource deletion on overflow.

• Do not replace broken logic with cosmetic animation.

02. Vision, Player Role, Objectives, and Core Loop

Player role

The player is the operator and strategic partner to the station AI. The station is the persistent base and main character-like entity. The player sets priorities, configures machines, purchases upgrades, sends the submarine on missions, and responds to warnings. The intended feeling is tending a living industrial organism, not clicking disconnected menus.

Core loop

1. Observe power, inventories, machine queues, station condition, alerts, and sub status.

2. Identify the bottleneck: power, storage, organic input, processing, research, repair, or exploration.

3. Set operating priorities or configure the next sub mission.

4. Advance simulation time; machines consume inputs, produce outputs, wear, and pause when blocked.

5. Sub travels, scans, mines/captures, manages battery/cargo/hull, and returns when safety rules trigger.

6. Unload cargo into finite storage; processing/refining convert materials into useful products.

7. Spend resources on repairs, modules, upgrades, and research.

8. Unlock deeper zones and harder deposits; repeat with more capability and risk.

Objectives and failure philosophy

• Short-term: keep power/storage stable, complete a safe mission, prevent a critical module failure.

• Mid-term: establish resource chains and upgrade hull, battery, tools, cargo, and sensors.

• Long-term: exploit rare deposits, expand capability, automate routine decisions, sustain operations at depth.

• Warnings precede most serious failures. Alerts explain cause, consequence, and response.

03. Visual Composition and Station Cutaway Architecture

Frame and spatial rules

• Station occupies upper portion; waterline is a clear horizontal boundary; underwater descent fills most remaining view.

• Facility shell is a continuous silhouette with visible plating, support ribs, and thickness. Rooms exist inside the shell.

• Floor slabs line up. Bulkheads terminate at floors/ceilings. Doors and hatches appear in plausible openings.

• Equipment has footprints and service clearance. No machine clips through walls or floats without a mount.

• Pipes connect compatible fluid ports; cables connect power sources and loads; transfer paths connect relevant modules.

• Damaged equipment shows state-dependent leaks, smoke, sparks, warning lamps, or disabled panels.

• The sub is small and readable. Terrain scrolls; sub remains easy to locate. Depth meter stays fixed on screen edge.

ZONE	PURPOSE	VISIBLE ELEMENTS	SYSTEM LINKS
AI Core/control	Control policies and diagnostics	Core cabinet, lamps, cable trunks	Power, sensors, comms, modules
Power room	Generate/distribute electricity	Generator, breaker panel, cooling pipes	Core, dock, powered machines

ZONE	PURPOSE	VISIBLE ELEMENTS	SYSTEM LINKS
Storage bay	Finite inventory	Racks, bins, sample cases, tanks	Dock, processor, refinery
Organic intake	Receive/sort biomass	Intake chute, capture basket, sorter	Organic storage, processor
Processing	Make bio-nutrients	Vats, filters, pumps, pipes	Organic feedstock, power, output
Refinery	Convert minerals/salvage	Reactors, tanks, output bins	Raw store, power, components
Research lab	Unlock technologies	Samples, instruments, bench	Power, storage, research
Comms/sensors	Monitor ocean and sub link	Antenna, sonar, relay	Core, sub, world
Sub dock	Launch/recover/charge/unload	Cradle, winch, charging port	Power, storage, maintenance

Layering and art direction

• Foreground hull/equipment gets strongest contrast; background uses limited parallax.

• Use reusable underwater tiles, not a giant unique scene. Center water remains mostly open.

• Interactive nodes have recognizable silhouettes and selection highlights; decorative coral/fish must not masquerade as resources.

• Ambient animation is restrained: bubbles, water drift, blinking lamps, machine movement, occasional fish and thrusters.

• No photorealism, no full 3D perspective, no wall of pipes across open underwater scene.

Readability hierarchy

At phone scale, identify station status, sub depth/location, current target, critical warnings, and primary action at a glance. Detailed engineering information belongs in inspection views, not permanent tiny labels.

04. Coordinates, Simulation Clock, and Update Order

Units

Use one consistent internal unit set: distance meters, time seconds, energy kWh, power kW, mass kg or configured resource units, temperature °C, pressure kPa, condition 0–100%. Do not mix rates and totals. Round for display only.

Simulation clock

Rendering and simulation are separate. Recommended fixed simulation step $\Delta t = 1$ simulated second. Rendering may run at 30–60 FPS. Pause freezes simulation timers while UI remains interactive. A speed multiplier scales simulation advancement uniformly.

Update order each tick

1. Apply queued player commands; validate costs and requirements.

2. Calculate generation, battery charge/discharge, and power allocation.

3. Resolve transfers and reserve inputs for machine jobs.

4. Advance processing, refining, capture, research, repairs, and construction.

5. Advance sub movement, depth, pressure, tool use, battery, cargo, and navigation.

6. Update world nodes, depletion, fish populations, and hazards.

7. Apply condition wear, heat, faults, and failure checks.

8. Recompute inventory/capacity, alerts, unlocks, and derived status.

9. Publish authoritative state and save at configured checkpoints.

Core formulas

```
attempted_quantity = rate_per_second * delta_seconds

actual_output = min(attempted_quantity * efficiency, input_limit, power_limit, free_output_capacity)

condition_factor = clamp(condition / 100, minimum_efficiency, 1.0)

effective_rate = base_rate * condition_factor * upgrade_multiplier * operating_modifier
```

For a simple prototype, minimum_efficiency may be 0.25 until condition reaches zero, at which point the machine is disabled. All values are configurable. Never allow negative inventory or condition.

Pause/speed

• Pause freezes all simulation systems, not just visible animation.

• Normal = 1×; optional fast modes = 2× or 4× using the same simulation clock.

• Optional pause-on-critical setting is explicit; do not unexpectedly alter pause state.

• Changing speed must not complete jobs twice or break accounting.

05. Common Module Framework and State Machine

Every station module follows a shared contract so UI, simulation, upgrades, faults, and save/load remain consistent.

FIELD	TYPE / RULE	PURPOSE
id	Stable unique string	Persistent module identity
type	Configured enum	GENERATOR, STORAGE, PROCESSOR, etc.
level	Integer ≥ 1	Upgrade tier
condition	Float 0–100	Working condition
mode	OFF/STANDBY/RUNNING/PAUSED/FAULTED/DESTROYED	Operational state
inputs/outputs	Resource→quantity maps	Inventory and recipes
power_demand/granted	Rate values	Requested vs actual power
base_rate/efficiency	Configured numbers	Production rate modifiers
queue	List of jobs	Recipes/research/repair
faults	Codes + severity	Active faults
upgrade_state	Data map	Installed upgrades
last_tick	Timestamp	Debugging/consistency

CURRENT STATE	CONDITION	NEXT STATE
OFF	Player starts and dependencies valid	STANDBY
STANDBY	Valid job, inputs, power, capacity	RUNNING
RUNNING	No input/power or output full	STANDBY/PAUSED
RUNNING	Recoverable fault	FAULTED
FAULTED	Fault repaired and requirements met	STANDBY
ANY ACTIVE	Condition zero / destroyed	DESTROYED
DESTROYED	Rebuilt/replaced if allowed	OFF

Common operating rules

• Reserve inputs at job start or consume continuously; choose one policy consistently.

• If output cannot be stored, pause before consuming more input unless recipe explicitly has a buffer.

• Insufficient power follows one documented policy: proportional rate reduction or stop. Show granted power and reason.

• Wear increases from active work/stress, not just being switched off.

• Inspection shows status, condition, current job, inputs/outputs, power, faults, upgrades, actions.

• Every action validates conditions before spending resources. Failed purchase has zero cost.

06. Station Modules and Machine Behavior

6.1 AI Core

Central controller; not a resource generator. Maintains module state, evaluates warnings, runs user-defined priority rules, coordinates automation, and records diagnostics.

• Inputs: power, sensors, configuration. Outputs: control decisions, alerts, mission recommendations.

• Upgrades: more automation rules, diagnostics, sensor range, coordination efficiency.

• Failure: automation becomes limited/delayed; preserve manual controls where possible.

• UI: overall status, alerts, module list, automation policy editor.

6.2 Generator

```
power_generated = rated_power * condition_factor * fuel_factor * upgrade_multiplier
```

• Output never exceeds rated/safe output.

• Fuel-based generator consumes configured fuel per energy unit; other generator types need explicit sources/modifiers.

• High load increases heat/wear. Overload can create faults.

• Upgrades affect output, efficiency, reliability, fuel capacity, overload tolerance.

• Failure lowers power budget and raises a high-priority alert.

6.3 Power distribution

Treat distribution as a logical network. Every powered module requests a rate; the allocator grants power by stable priority. Suggested tiers: 0 = Core/emergency sensors; 1 = sub safety/dock; 2 = processing/refining; 3 = research/optional utilities.

```
available_power = generation + battery_discharge - required_reserve
grant_i = min(request_i, remaining_power_for_priority_tier)
```

Within a tier, use proportional allocation or stable module-ID order. Pick one and expose it in settings. When demand exceeds supply, lower-priority systems slow or pause. Show generated, requested, granted, and reserve power.

6.4 Storage

• Finite capacity measured in mass or volume; choose one authoritative rule.

• Separate classes: bins for solids, tanks for fluids, sealed cases for samples, chilled bins for perishable organics.

• Transfer limited by source amount, destination capacity, and transfer rate.

• Overflow pauses incoming transfers and shows warning; never delete resources silently.

• Optional spoilage only for configured perishable resources with explicit decay/byproduct rules.

```
transfer = min(requested, source_available, destination_free_capacity, transfer_rate * delta_seconds)
```

6.5 Organic capture/intake

• Receives fish, biomass, and other organics from a real configured source or sub cargo.

• Inputs: delivered organics; power for sorting/pumping. Outputs: sorted biomass, samples, waste, feedstock.

• Rate limited by equipment, input, power, and storage.

• Optional ecology: overharvest reduces local fish availability; recovery uses defined regeneration.

• Upgrades improve capture efficiency, sorting, throughput, or sustainable yield.

6.6 Processing

RECIPE EXAMPLE	INPUT	OUTPUT	TIME
Biomass → bio-nutrients	10 biomass	6 bio-nutrients + 2 waste	10 s
Fish → concentrate	5 fish	3 concentrate + 1 sample	12 s
Mixed organics → feedstock	8 mixed organics	5 feedstock + 2 waste	15 s

Example values only; recipes live in configuration. UI shows inputs, outputs/byproducts, duration, power, and exact blocked reason.

6.7 Refinery

RECIPE	INPUTS	OUTPUTS	NOTES
Scrap → metal stock	10 scrap	6 metal stock + 2 slag	Starter
Ore → refined mineral	8 ore + 2 bio-nutrient	5 refined mineral + 1 tailings	Catalyst required
Salvage → components	6 salvage	3 components + 1 scrap	Configurable salvage
Oil-bearing biomass → fuel/oil	8 biomass	4 fuel/oil + 2 waste	Suitable feedstock

6.8 Research lab

• Research is a timed queue with explicit cost, duration, prerequisites, and effect.

• Validate costs before start. Consume costs once at start; power loss pauses progress without reset.

• Completion unlocks technology and applies explicit effect.

• Tech branches: hull/depth, battery, tools, cargo, navigation, processing, storage, sensors, station efficiency, automation.

• Do not grant tech merely because a button was clicked.

6.9 Communications and sensors

```
link_quality = clamp(base_link - depth_penalty - obstruction_penalty + relay_bonus, 0, 1)
```

• Low link quality delays/limits noncritical remote commands; local safety rules still run onboard.

• Scans consume time/power; reveal position, composition confidence, and hazards.

• Show last contact depth, link quality, telemetry age.

• Upgrades improve range, relay strength, scan clarity, or confidence.

6.10 Dock, charge, unload, maintenance

• Dock states: APPROACHING, DOCKING, DOCKED, UNLOADING, CHARGING, MAINTENANCE, LAUNCH_READY.

• Cargo transfers only when docked and only up to storage capacity and transfer rate.

• Charging consumes station power and cannot exceed battery capacity.

• Repairs consume time and configured components/materials.

• Launch checks validate hull, battery, tools, mission, and safety; explicit override may allow a warned launch.

• Dock status is derived from submarine state, not a decorative toggle.

07. Resources, Inventory, Units, and Recipes

Every resource has stable ID, display name, category, unit, mass/volume, storage class, stack limit, spoilage rule, rarity, and allowed recipes.

ID	CATEGORY	STORAGE / UNIT	USE
biomass	Organic	kg / bin	Processing feedstock
fish	Organic	kg / chilled bin	Samples and processing
bio_nutrient	Processed organic	units / sealed bin	Catalyst/feedstock
mixed_ore	Mineral	kg / ore bin	Refining
refined_mineral	Processed mineral	units / bin	Advanced components
scrap	Salvage	kg / bin	Metal stock/components
metal_stock	Processed metal	units / rack	Construction/repairs
components	Manufactured	units / rack	Repairs/upgrades
fuel_oil	Fuel	L / tank	Fuel generator/processing

ID	CATEGORY	STORAGE / UNIT	USE
rare_sample	Research	sample / case	Research unlocks
waste	Byproduct	kg / waste bin	Recycling/disposal
slag_tailings	Byproduct	kg / bin	Optional recovery

Conservation rules

• Quantities are conserved through transfers/recipes except configured yields, byproducts, or explicit process losses.

• Every recipe accounts for outputs/byproducts. Lost mass is an explicit loss/heat/water term where useful.

• UI never keeps a separate fake inventory counter.

• Purchases/transfers are atomic: validate, then commit.

• Use sufficient precision internally; round display only.

```
Recipe = { id, duration_s, power_kw, inputs{}, outputs{}, byproducts{}, min_level, required_tech, heat_rate, wear_rate
}
```

08. Autonomous Submarine: State Machine and AI

One small autonomous deep-sea submarine with tiny articulated arms. It scans, grabs, mines, and captures at resource nodes on cliff faces and seabed. It is not human-piloted. Local safety rules work even when communications are lost.

STATE	BEHAVIOR	EXIT CONDITION
DOCKED	Unload, charge, repair, configure	Launch + checks pass
PRE_LAUNCH_CHECK	Validate hull, battery, tools, cargo, target	Pass→DEPARTING; fail→DOCKED with reason
DEPARTING	Leave dock safely	Clear of dock
TRAVELING	Move to waypoint; consume battery	Target reached→SCANNING
SCANNING	Reveal node and hazards	Target chosen or scan done
APPROACHING_NODE	Navigate and align tool	Within tool range
WORKING_NODE	Mine/grab/capture; energy and wear	Depleted/full/hazard/complete
TRANSIT_TO_NEXT	Choose next valid target	Reached or return trigger
RETURNING	Navigate to dock safely	Dock approach
DOCKING	Align and connect	Docked
EMERGENCY_RETURN	Abort optional work; prioritize return	Docked or disabled
DISABLED	Cannot safely move/use tool	Recovery/rescue

Mission configuration

• Target: selected node, region, or depth band.

• Mode: SCAN, MINE, SALVAGE, CAPTURE ORGANICS, MIXED COLLECTION, EXPLORE.

• Priority: high-value, nearest, safest, player-selected order.

• Return triggers: cargo threshold, battery reserve, hull threshold, tool condition, mission complete, manual command.

• Safety reserve, maximum depth, optional maximum mission duration configurable.

Movement

```
distance = sqrt((x_target - x)^2 + (depth_target - depth)^2)

travel_time = distance / max(effective_speed, minimum_speed)

effective_speed = base_speed * thruster_condition * load_factor * current_modifier
```

Suggested load_factor = max(0.55, 1 - 0.35 * cargo_fraction): full cargo slows the sub by up to 35%.

Battery

```
energy_used_kWh = total_power_kW * delta_seconds / 3600
return_reserve = estimated_return_energy * safety_multiplier + emergency_reserve
```

Suggested safety multiplier = 1.25. At reserve, stop optional work and enter EMERGENCY_RETURN. This local rule applies even without station contact.

Mining tool

• Tool has tier, power draw, rate, range, allowed hardness, heat, and condition.

• Work requires revealed/reachable node, range, alignment, and sufficient tier.

• Mining decreases node quantity and increases cargo by the same amount, limited by node and capacity.

• Tool use increases heat/wear; overheating pauses work; low condition reduces rate.

• Weak tools cannot mine every deposit.

```
mining_rate = base_rate * tool_condition_factor * tier_bonus / max(1, hardness)
mined_amount = min(mining_rate * delta_seconds, node_remaining, cargo_free_capacity)
```

Cargo

```
cargo_used = sum(quantity_i * mass_per_unit_i)
cargo_fraction = clamp(cargo_used / cargo_capacity, 0, 1)
```

• Suggested return threshold 85%; finish current safe action, then return.

• At full capacity, stop collection and return.

• Cargo stays aboard until docking; do not duplicate it in station storage early.

• If station storage is full, remaining cargo stays aboard and UI shows reason.

• Bulky salvage and light rare samples may use different mass/volume.

Autonomous target scoring

```
score(node) = value_weight*normalized_value + priority + proximity_bonus - risk_weight*risk -
energy_weight*estimated_energy
```

Filter candidates first: reachable, within depth rating, tool-compatible, not depleted, cargo-compatible, and feasible within battery reserve. Choose highest score with deterministic tie-break by node ID. If no valid target, return; never loop on unreachable targets.

Return and emergency rules

• Return on cargo threshold, battery reserve, hull threshold, broken tool, mission completion, or explicit order.

• Resolve immediate collision avoidance before route change.

• Replan at fixed intervals, not every render frame. Never teleport to dock.

• If no safe path exists, enter safe holding state, raise alert, try alternate routes.

• Docking, unloading, charging, and maintenance are separate states with real progress.

09. Underwater World, Depth, Pressure, Hazards, Deposits

World structure

• Vertical side-view environment with stable visible region around sub and scrollable terrain/background.

• Left/right cliff/reef edges provide ledges and resource attachment points; center water stays open.

• Seabed has sediment, wrecks, vents, minerals, and unusual samples.

• Fish/organic sources are limited populations or defined nodes; decorative fish are not automatically collectible.

• Use seeded placement and reusable tiles; do not create one giant unique level.

Pressure

Use depth in meters and seawater approximation rho=1025 kg/m³, g=9.81 m/s², atmospheric pressure=101.3 kPa.

```
pressure_kPa = 101.3 + (1025 * 9.81 * depth_m / 1000)
pressure_atm = pressure_kPa / 101.3
```

At 100 m, pressure is roughly 1,107 kPa absolute. Each sub has rated pressure. Warn when margin is low; block descent beyond rating unless a deliberate risk override exists.

```
pressure_margin = rated_pressure_kPa / current_pressure_kPa
overpressure_ratio = max(0, current_pressure / rated_pressure - 1)
hull_damage_per_s = base_overpressure_damage * overpressure_ratio^2 * stress_multiplier
```

The squared model makes small exceedances survivable but large exceedances dangerous. It is a game model, not a real engineering certification.

Temperature, visibility, communications

```
visibility = clamp(base_visibility - depth_penalty - turbidity_penalty + sensor_bonus, min_visibility, max_visibility)
```

- Low visibility reduces scan radius and target confidence.
- Temperature modifies battery efficiency/tool cooling/wear only through explicit rules.
- Show depth, pressure, temperature, visibility, hull, and link quality in sub inspection.
- Do not change stats without a rule or explainable source.

DEPTH BAND	ENVIRONMENT	RESOURCES	REQUIREMENT
Surface/shelf	Brighter, mild pressure	Fish, biomass, scrap, common ore	Starter sub
Upper slope	Dimmer, moderate pressure	Mixed ore, salvage, organics	Hull upgrade useful
Deep slope	Cold, low visibility	Rare minerals, wreck parts, samples	Hull/battery/tools
Abyssal	Very dark, high pressure	High-value deposits, specialized samples	Advanced upgrades
Extreme depth	Severe pressure, sparse visibility	Endgame materials and rare events	Top-tier gear/reserves

Node model

```
ResourceNode = { id, x, depth_m, type, composition, remaining_quantity, hardness, hazards, revealed, regeneration_rate }
```

- Scanning reveals position, composition confidence, and hazards.
- Mining reduces remaining_quantity; zero means depleted.
- Renewable nodes recover only if regeneration_rate > 0.
- Fish populations recover over time only by defined ecology rules.
- Node contents deterministic for a seed and ID unless an explicit random event is configured.

10. Economy, Progression, Upgrades, Difficulty

Economy

- Every upgrade defines cost, prerequisite, target module, install time, and exact field effect.
- Validate all costs before deduction; no partial purchase.
- Research unlocks capability; resources pay for installation; module levels improve performance.
- Avoid infinite profit loops unless intentionally designed as a progression reward.
- Resource sinks: repairs, construction, research, tool replacement, module upgrades, maintenance.

CATEGORY	EXAMPLES	EFFECT
Sub hull	Reinforcement, pressure joints	Higher pressure rating; less stress damage
Battery	High-density cells	More energy, longer missions
Propulsion	Efficient thrusters	Higher speed or lower energy/distance
Tools	Tiered cutter/drill/arm	Harder materials, higher rate

CATEGORY	EXAMPLES	EFFECT
Cargo	Larger/compact hold	More capacity
Sensors	Sonar, analyzer	Range, composition knowledge
Automation	Target scoring/safety rules	Smarter routing and return
Station power	Generator/distribution tier	More output and reliability
Processing	Filter/reactor upgrades	Yield, rate, less waste
Storage	Bins/tanks/racks	Capacity and transfer rate
Research	Lab equipment	Unlock advanced technologies
Maintenance	Diagnostics/spares	Less downtime and repair cost

Upgrade contract

```
Upgrade = { id, cost, prerequisites, target_system, effects:[{field, operation, value}], max_level, install_time_s }
```

Every effect names the exact field it changes. Example: Hull Reinforcement Tier 2 increases rated_pressure_kPa by 30%. UI displays before/after values.

Progression gates

• Deeper zones require hull rating for that pressure range.

• Hard deposits require sufficient tool tier and power.

• Advanced recipes require research and correct machine level.

• Rare samples unlock deep exploration and station resilience tech.

• Introduce one new constraint at a time, then combine systems later.

Difficulty

• Increase constraints, travel cost, resource complexity, visibility, and environmental risk—not arbitrary health multipliers.

• Early: shallow resources, short missions, simple power and recipes.

• Mid: power competition, cargo planning, tool tiers, storage bottlenecks, pressure.

• Late: rare samples, expensive repairs, complex refining, long trips.

• Events need telegraphing and defined duration/effect.

11. Damage, Wear, Maintenance, Faults, Cascades

```
condition_next = clamp(condition_current - base_wear_rate*activity_factor*stress_factor*delta_seconds, 0, 100)
```

Suggested thresholds: 70% minor warning, 40% serious warning, 15% critical warning. Condition 0 disables component.

FAULT	TRIGGER	EFFECT	RECOVERY
Generator overheating	High load/low condition	Output drops, wear rises	Reduce load, cool, repair
Power branch fault	Stress/damage	One branch loses power	Repair distribution
Storage jam	Overload/low condition	Transfer rate reduced	Maintenance
Processor clog	Dirty feedstock/wear	Rate/yield reduced	Cleaning cycle
Refinery thermal fault	Heat/low condition	Recipe pauses	Cool and repair
Tool jam	Hard material/low condition	Mining stops	Repair at dock
Navigation fault	Damage/uncertain sensor	Route planning slows	Replan/repair
Comms loss	Depth/obstruction/relay damage	Telemetry delayed	Return or relay upgrade
Cargo transfer blocked	Storage full	Unload pauses	Free capacity
Hull over-stress	Pressure beyond rating	Hull damage rises	Emergency return/repair

Example cascade

1. Generator condition falls; output declines.

2. Power allocator protects Core and dock; refinery receives less power.

3. Refining slows; components arrive later.

4. Sub returns with salvage but repairs are delayed by component shortage.

5. Next trip is riskier; player reduces optional loads, repairs generator, or waits for materials.

Alerts identify root cause and affected systems, not just 'system failure'. Cascades are fair only when the player can understand and respond.

12. UI, Interaction, Alerts, Accessibility

Main screen

• Top strip: generation/demand, Core condition, storage usage, key resources, simulation day/time if used.

• Station cutaway: hotspots align to visible machines.

• Underwater viewport: open descent, sub, terrain, nodes, fish, wrecks, subtle effects.

• Fixed depth meter with ticks, current marker, target depth, safe limit.

• Sub panel: state, depth, hull, battery, cargo, tool condition, mission, return reserve.

• Compact pause/speed, return, upgrades, research, build, alerts controls.

Inspection panel

AREA	REQUIRED CONTENT
Header	Name, type, level, state, condition
Summary	Purpose and current status
Inputs/outputs	Quantities, required amounts, rates, blocked reasons
Power	Requested, granted, consumed/produced
Work	Current job, progress, remaining time, queue
Condition	Wear, faults, diagnostics
Actions	Start/stop, priority, recipe, repair, upgrade, connections
Consequences	Cost and before/after values before confirmation

Interaction rules

• Every action has enabled/disabled/pending/success/failure state.

• Disabled actions show reason: insufficient components, no power, missing prerequisite, target unreachable, storage full.

• Confirm expensive irreversible actions and destructive overrides.

• UI updates from state model after action; no manual fake visual result.

• Progress bar represents real job and distinguishes running/paused/blocked.

• Critical alert includes cause, consequence, response.

Accessibility

• Readable labels at phone scale; do not use color alone.

• Large touch targets; pause always accessible.

• Reduced-motion option where feasible.

• Advanced numbers expandable; main screen stays calm.

13. Data Model, Pseudocode, Validation

Simulation state is authoritative and separate from rendering. UI reads snapshots/view models; it never invents values.

```
GameState = { schema_version, simulation_time_seconds, speed_multiplier, paused, world_seed, station: { modules, inventories, power_network, research, alerts }, submarine: SubState, world: { depth_bands, resource_nodes, fish_populations, hazards }, progression: { unlocked_technologies, purchased_upgrades, milestones }, settings: UserSettings }

ResourceDefinition = { id, display_name, category, unit, mass_per_unit, volume_per_unit, storage_class, stack_limit, perishable, decay_rate_per_second, tags }

ModuleDefinition = { id, display_name, footprint, max_level, base_power_kw, base_rate, recipes, wear_rate_per_second, required_technology }
```

Power allocator pseudocode

```
function allocatePower(modules, generation, battery, dt): available = generation + battery.maxDischargeFor(dt) sort loads by priority ascending, then stable module ID for load in loads: request = load.requestedPower() grant = min(request, available) load.setPowerGranted(grant) available -= grant update battery discharge from actual supplied energy if generation exceeds demand and battery accepts charge: charge battery from remaining generation publish generated, demanded, granted, reserve
```

Machine tick pseudocode

```
function tickMachine(machine, dt): if mode in [OFF, DESTROYED, FAULTED]: update passive cooling; return if powerGranted < minimumOperatingPower: mode = STANDBY; reason = "Insufficient power"; return if no valid recipe: mode = STANDBY; return if inputs unavailable: mode = STANDBY; reason = "Missing input"; return if output capacity unavailable: mode = STANDBY; reason = "Output full"; return mode = RUNNING advance progress using effective rate; consume power; apply heat/wear if job complete: atomically consume reserved inputs and add outputs/byproducts clear job; start next valid queued job
```

Submarine decision pseudocode

```
function decideSub(sub): update local sensors and safety estimates if hull critical or battery <= returnReserve: state = EMERGENCY_RETURN if state == DOCKED: wait for launch elif state in [TRAVELING, SCANNING, WORKING_NODE]: if cargoFraction >= returnThreshold or missionComplete or noValidTargets: state = RETURNING else continue mission safely elif state in [RETURNING, EMERGENCY_RETURN]: plan safe route to dock if no route: safe holding state + alert if state == DOCKED: unload up to station capacity charge up to battery capacity and available power repair only with materials and maintenance capacity
```

Determinism

Use seeded random generation for world placement, optional events, and faults. Save seed/RNG state. Same save plus same command sequence should reproduce the same simulation. Never use render-frame timing as gameplay randomness.

14. Save/Load, Determinism, Performance, Mobile

Save system

• Save authoritative state: inventory, module conditions/modes/queues, research progress, sub position/state/cargo/battery, node depletion, upgrades, alerts, simulation time, settings, RNG state.

• Use schema_version and migrations. Validate or repair invalid state instead of crashing.

• Autosave at safe intervals and milestones: research completion, docking, unloading, upgrades.

• Atomic write: temporary file, validate, replace prior save. Keep last valid backup.

• Do not serialize transient UI animation as gameplay truth.

• On load, clamp condition, validate IDs, ensure inventory nonnegative, reconcile jobs.

Performance

• Target stable 60 FPS where possible and 30 FPS on lower-end phones.

• Pool particles/effects; cap bubbles, sparks, fish, debris.

• Reuse background tiles and parallax layers; no huge unique world image.

• Keep simulation lightweight; do not recalculate every node each render frame.

• Animate nearby fish; distant layers can be static/slow.

• Profile on real mobile hardware early.

Offline/error handling

• Core game is single-player/offline; no server dependency.

• Malformed definitions fail validation in development rather than silently becoming free recipes.

• Missing art can use a safe placeholder without breaking logic.

• Invalid commands return a visible reason and do not mutate state.

• Save corruption recovery uses last valid backup if available.

• Log simulation time, system ID, and relevant state on error.

15. Example Starting Values and Balancing Guardrails

Concrete prototype defaults, not final balance. Keep editable in configuration.

PARAMETER	START VALUE	NOTES
Simulation tick	1.0 s	Fixed deterministic step
Initial generation	50 kW	One starter generator
AI Core demand	3 kW	Always-on
Sub dock/charging	0–12 kW	Varies with charge rate
Basic processor	8 kW	Creates power trade-off
Basic refinery	10 kW	Competes with research
Sub battery	100 kWh	Short early missions
Sub speed	2.0 m/s	Tune mission length
Cargo capacity	100 kg	Encourages returns
Return safety multiplier	1.25	25% energy margin
Basic tool rate	0.5 kg/s	Hardness/wear reduce rate
Starter hull rating	1,200 kPa abs.	Shallow/some slope
Minor condition warning	70%	Maintenance notice
Serious warning	40%	Repair prompt
Critical warning	15%	High-priority alert
Cargo return threshold	85%	Leaves buffer
Storage overflow	Pause transfer	Never delete resources
Research interruption	Pause/retain progress	Power loss recoverable

Economy guardrails

• Early successful trip funds one modest improvement, not every upgrade.

• Repairs affordable if return happens before critical damage; emergency recovery costs more.

• Rare materials motivate deeper trips but are not required for basic survival.

• Power/storage become meaningful constraints before they become chores.

• Every bottleneck has at least one clear upgrade or strategy.

Balance telemetry

• Average mission duration/return rate.

• Emergency returns per play hour.

• Times storage overflow blocks production.

• Time machines are paused by power shortage.

• Waiting time vs meaningful decision time.

• Most-used/never-used upgrades.

• Failure causes and whether alerts were seen beforehand.

16. Test Plan and Acceptance Criteria

ID	TEST	PASS CONDITION
SIM-01	Advance 10,000 ticks	No negative inventory/invalid condition/crash
SIM-02	Pause 60 real seconds	Simulation time and progress unchanged
SIM-03	Run at 2×	Twice sim advancement; no duplicate completions
RES-01	Transfer 20 from source holding 12	At most 12; source never negative
RES-02	Fill storage	Transfer pauses with clear reason
PWR-01	Demand exceeds generation	Priority consistent; rates reflect grant
MCH-01	Recipe lacks input	No output; reason shown
MCH-02	Output fills mid-job	Pause/buffer follows explicit policy
SUB-01	Cargo reaches threshold	Safe action ends; sub returns
SUB-02	Battery reaches reserve	Optional work stops; return begins
SUB-03	Tool too weak	Mining blocked with explanation
SUB-04	Storage full at docking	Unload pauses; cargo stays aboard
DEP-01	Increase depth	Pressure rises using formula
DEP-02	Exceed hull rating	Warning/damage follow rule
UI-01	Select module	Panel shows authoritative live values
SAVE-01	Save mid-mission	Position/state/cargo/battery/target preserved
SAVE-02	Save mid-recipe	Progress/reserved inputs preserved
VIS-01	Small phone display	Station/depth/sub/alerts/controls readable

Visual acceptance

• Station is coherent cutaway, not floating buildings.

• Floors align; equipment fits; no hull clipping.

• Visible pipes/cables connect logically.

• Station above waterline; open ocean below.

• Central water lane clear; sub easy to find.

• Depth meter fixed and follows actual depth.

• Interactive nodes differ from decoration.

• A complete launch→collect→return→unload→process→upgrade cycle can be watched.

End-to-end scenario

1. Start with Core, generator, finite storage, processor, refinery, dock, sub.

2. Launch shallow mixed-collection mission.

3. Sub scans, approaches, mines, updates cargo and consumes battery.

4. Cargo threshold triggers return without teleporting.

5. Docking/unloading changes station inventory by exact amount.

6. Processor/refinery consumes inputs, power, and time; outputs/byproducts appear.

7. Buy hull/cargo upgrade; costs deducted once and stats change.

8. Save/reload preserves state.

9. Repeat with low power and full storage; machines pause with clear reasons.

17. Milestones and Scope Control

MILESTONE	IMPLEMENT	DONE WHEN
M0 Visual blockout	Cutaway, waterline, open descent, depth meter, placeholder sub	Composition matches; station feels physical
M1 State foundation	GameState, fixed tick, pause/speed, save/load, resources	State persists; sanity tests pass
M2 Station machines	Core, generator, storage, processor, refinery	Power/input/output/condition work
M3 Sub mission	Movement, scan, mine, cargo, battery, return, dock	Full mission completes without fake state
M4 Economy	Research, upgrades, repair costs/effects	Validation and stats correct
M5 Depth world	Depth bands, pressure, deposits, visibility, hazards	Depth affects capability/risk
M6 UI	Inspection panels, alerts, mission config, queues	Values live; actions work
M7 Art	Cutaway art, machine motion, ocean effects	Readable on phone and on-style
M8 QA	Tune rates, acceptance tests, profile	No critical bugs/performance spikes

Scope discipline

• Build one complete vertical slice before expanding content.

• Prefer a few functioning machines over many decorative modules.

• Do not add crew, hunger, human oxygen, living quarters, combat, or online systems unless scope changes.

• Avoid fluid dynamics/3D physics if simpler deterministic rules deliver the fantasy.

• At each milestone, test and capture a screenshot before expanding.

18. Exact Instructions for the AI Game Creator

Follow these requirements in order. This is an implementation task, not a request to invent a different game.

1. Inspect current project, engine, code, assets, and existing mechanics before replacing anything. Preserve useful code.

2. Build composition first: physical station cutaway above, clear waterline, open underwater side-view below, fixed depth meter, small autonomous sub. Never turn ocean into a top-down map.

3. Rebuild station artwork as one coherent facility: continuous hull, rooms, floors, bulkheads, access paths, machinery footprints, pipes, cables, believable equipment. Aim for Fallout-style shelter cutaway readability, not clickable buildings.

4. Separate simulation state from rendering. UI reads authoritative state; it does not fabricate production/progress.

5. Implement fixed-step simulation and data-driven resource catalogue before adding many machines.

6. Implement real generator, finite storage, processor, refinery, AI Core, and dock. Verify every input, output, power flow, rate, and condition.

7. Implement sub state machine: travel, scan, target, mine/grab/capture, battery use, cargo/depth limits, return, dock, unload, charge, repair.

8. Make pressure, hull rating, tool tier, cargo, battery, power, and condition mechanically real.

9. Implement recipes, research, upgrades, wear, faults, and alerts as data-driven systems with explicit costs/effects.

10. Build contextual inspection panels showing live status, condition, inputs/outputs, power, current job, faults, and actions.

11. Add remaining systems only after the vertical slice passes end-to-end test.

12. Run all acceptance tests in Section 16. Fix failures rather than hiding them with visual workarounds.

13. At each milestone report what works, what remains, assumptions, and known bugs. Never claim completion unless state changes are implemented and tested.

Required output

• Playable prototype, not just a mockup.

• Reusable module definitions and clear project structure.

• Data definitions for resources, recipes, modules, upgrades, depth bands, and node types.

• Authoritative simulation tick/state model.

• Functional save/load with schema version.

• Inspection UI with clear blocked reasons.

• Acceptance-test results and known limitations.

Definition of done

Player can start a game, understand the cutaway, launch the submarine, collect a real resource from a finite node, return safely, unload into finite storage, process it using power and time, research or buy an upgrade, observe the upgrade changing actual behavior, save, reload, and continue. A visual mockup alone is not complete.

Final north star

The station should feel like a believable machine that happens to be a place. Every room, pipe, module, resource, and action either communicates a real system or is clearly decorative. The ocean feels open and deep. The sub operates autonomously but obeys the same resource, power, damage, and pressure rules as everything else. The player can understand cause and effect.

END OF SPECIFICATION • AI STATION • EXTREME IMPLEMENTATION BRIEF